

Document Intelligence Platform

- Financial document extraction, validation and review
- Scope: invoice, balance sheet, profit & loss, cash flow
- Submission status: locally verified; completeness and deployment pending

Problem and scope

- Accept PDF, JPG and PNG, including scanned documents, at most 3 pages
- Extract meaningful fields, tables and comparative-period values
- Return null for missing data and evidence for supported values
- Make calculations and stored results inspectable through UI and API

Architecture

- Browser upload -> FastAPI -> file validation
- PyMuPDF native text / Tesseract OCR -> configurable LLM
- Pydantic and evidence checks -> Decimal financial checks
- SQLAlchemy -> SQLite locally / PostgreSQL in deployment
- Dashboard and document-name API read the same persistent record

Input validation and OCR

- Check extension, content signature, byte size, image decode and PDF render
- Reject corruption, encryption, repaired PDFs and more than 3 pages
- Native text where adequate; render image-dominated pages for OCR
- EXIF correction and orientation detection; bounded OCR runtime
- Supplied inventory: 30 PDFs and 20 JPEGs; most PDFs need OCR

Structured extraction and grounding

- User selects document type; no classification service
- Dynamic fields, full table rows and separate comparative periods
- JSON schema plus server-side validation; reject truncated responses
- Source excerpt and page number accompany values
- Clear values lacking evidence; OCR evidence is not proof of accuracy

Financial validation

- Invoice: quantity x price, line sum, tax-inclusive/exclusive total and cash/change
- Balance sheet: assets vs capital/liabilities and complete component sums
- P&L: banking income, expenditure, minority interest and appropriation
- Cash flow: activities plus FX; opening plus net change and adjustments
- Decimal arithmetic; absolute 0.02 tolerance; brackets are negative
- Missing operands -> NOT_APPLICABLE; each period checked separately

API, persistence and frontend

- POST /api/v1/documents/process; GET /api/v1/documents/{document_name}
- GET /api/v1/documents; GET /api/v1/health; Swagger at /docs
- Atomic upsert: latest completed result per document name
- Searchable database dashboard, fields, tables, validation and raw JSON
- Clear missing/failure highlights; controlled error messages

Testing and actual verification

- Automated file, arithmetic, grounding, provider-error and API tests
- Full API test uses real PDF/SQLite and explicitly mocked LLM
- Local HTTP: frontend/assets, docs, health and listing verified
- Stored supplied results: all four types, Gemini 2.5 Flash with local OCR
- Sample outputs distinguish stored results, historical errors and synthetic checks
- Browser desktop/mobile verified; no deployed result is claimed

Deployment plan

- One Render Python service hosts API and frontend; no Docker
- Environment-only provider key, URL, model and PostgreSQL connection
- Build installs Python dependencies and local Tesseract Debian packages
- Linux package resolution and PostgreSQL require deployment validation
- Publishing and deployment deferred; all public URL entries pending

Limitations and next steps

- Resolve missing statement tables and documented cash-flow OCR error
- Run supplied samples and review every field, table and equation
- OCR errors, unfamiliar layouts and provider output limits remain risks
- Add production authentication, retention, limits, backups and evaluation
- Candidate: review code, run tests, rehearse demo and approve submission